

STRIPE PROJECT

8. Global Data Architecture

D1. Data Architecture Diagrams

Nicolas Pichon - AIA RNCP 38777 / BC02 / D1 : Data Architecture Diagrams / v28 - 2026/10/13.

D1.1. Contexte

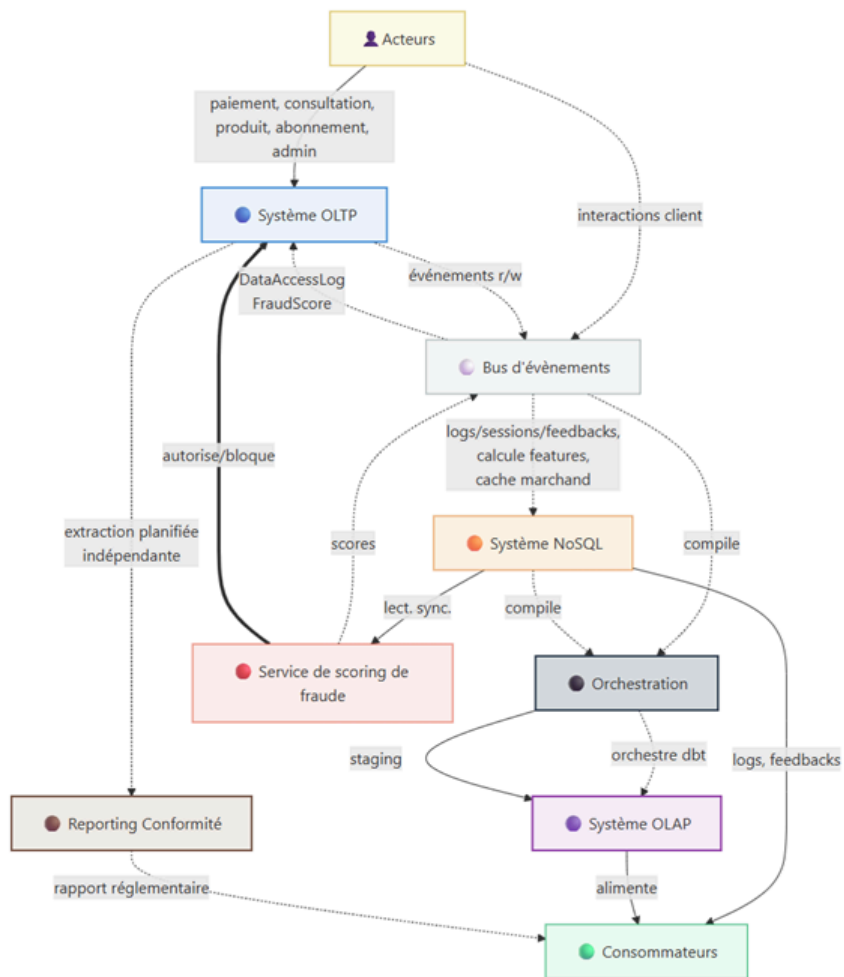
Ce document présente l'architecture de données du *business case* [R0]. Il décrit l'intégration des systèmes *OLTP* [D2], *OLAP* [D3] et *NoSQL* [D4], et détaille les flux de données et les pipelines qui lient ces systèmes

Quelques principes particuliers structurent l'architecture :

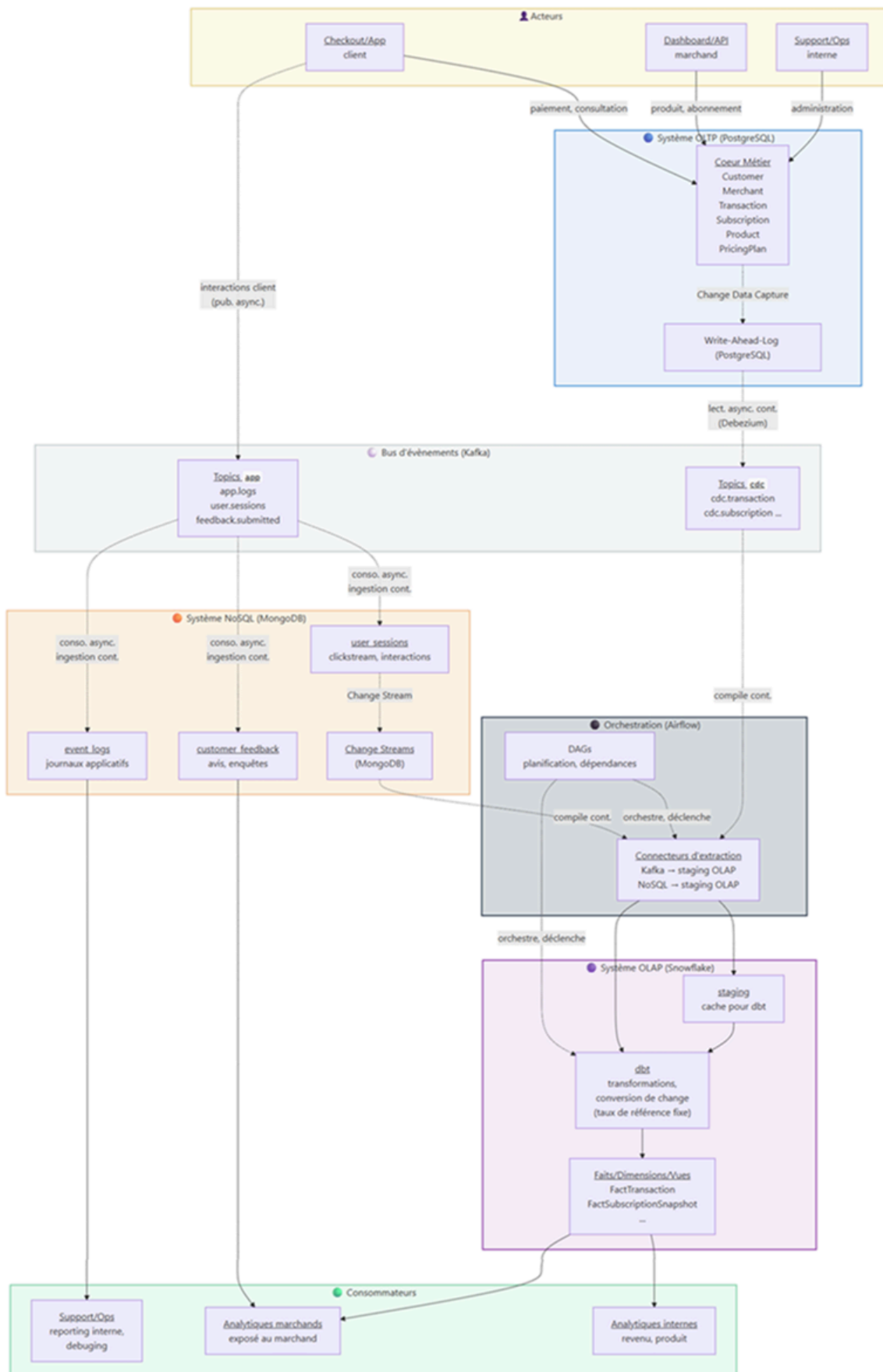
1. *Le chemin transactionnel ne doit jamais être ralenti* par des processus de sécurité, de conformité, d'analytique ou de détection de fraude. Les flux de données qui partent du système transactionnel vers les autres systèmes sont donc asynchrones.
2. *La réplication des changements d'état OLTP ne doit pas être synchrone*. On évite les déclencheurs synchrones (*triggers*) et on privilégie l'exploitation asynchrone du flux *CDC* (*Change Data Capture*) natif du système *OLTP*.
3. *Le taux de change est fixe* dans la base analytique. Il est actualisable périodiquement (annuellement par exemple).

D1.2. Diagramme d'architecture globale des données

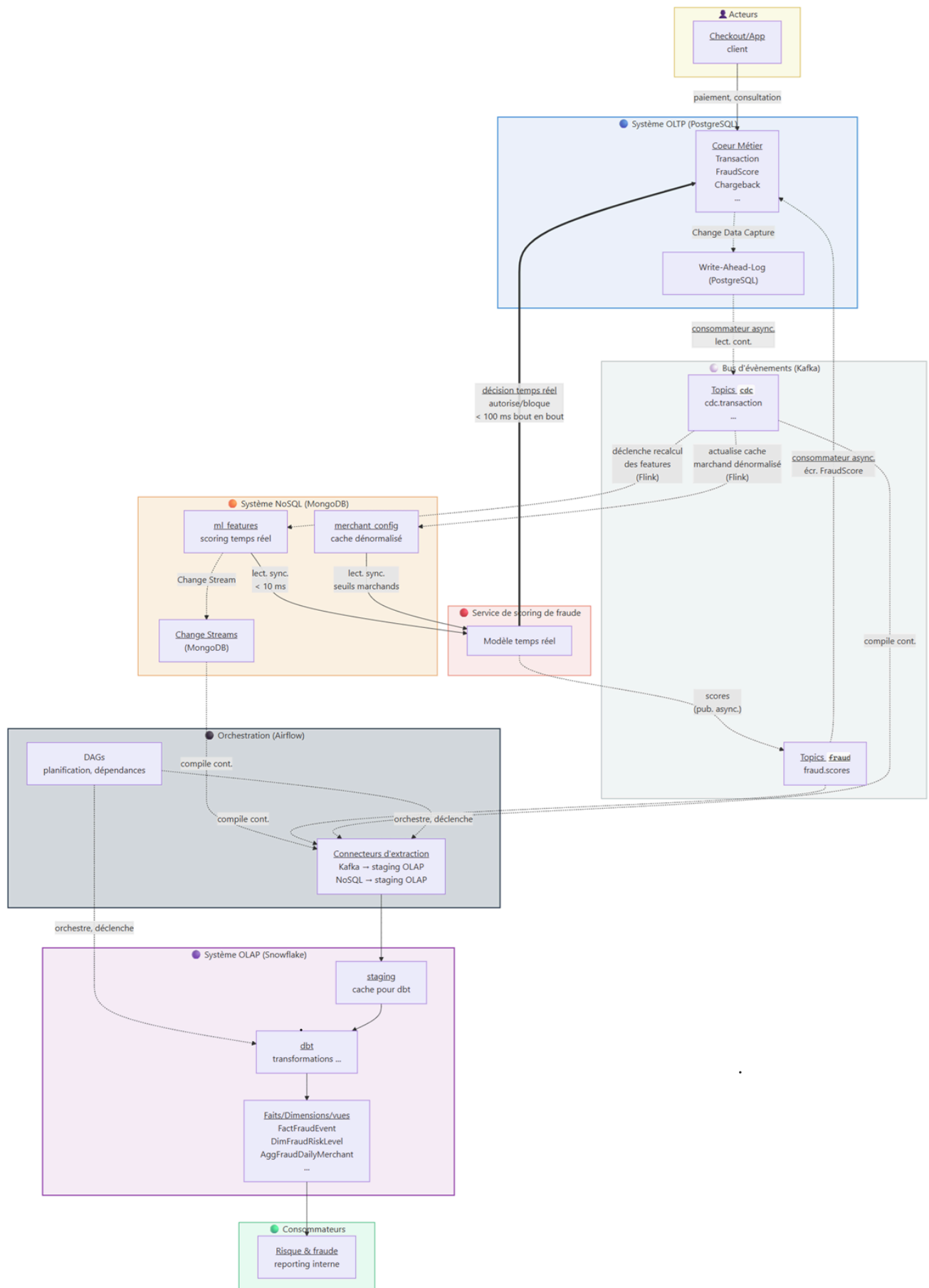
D1.2.1. Vue globale



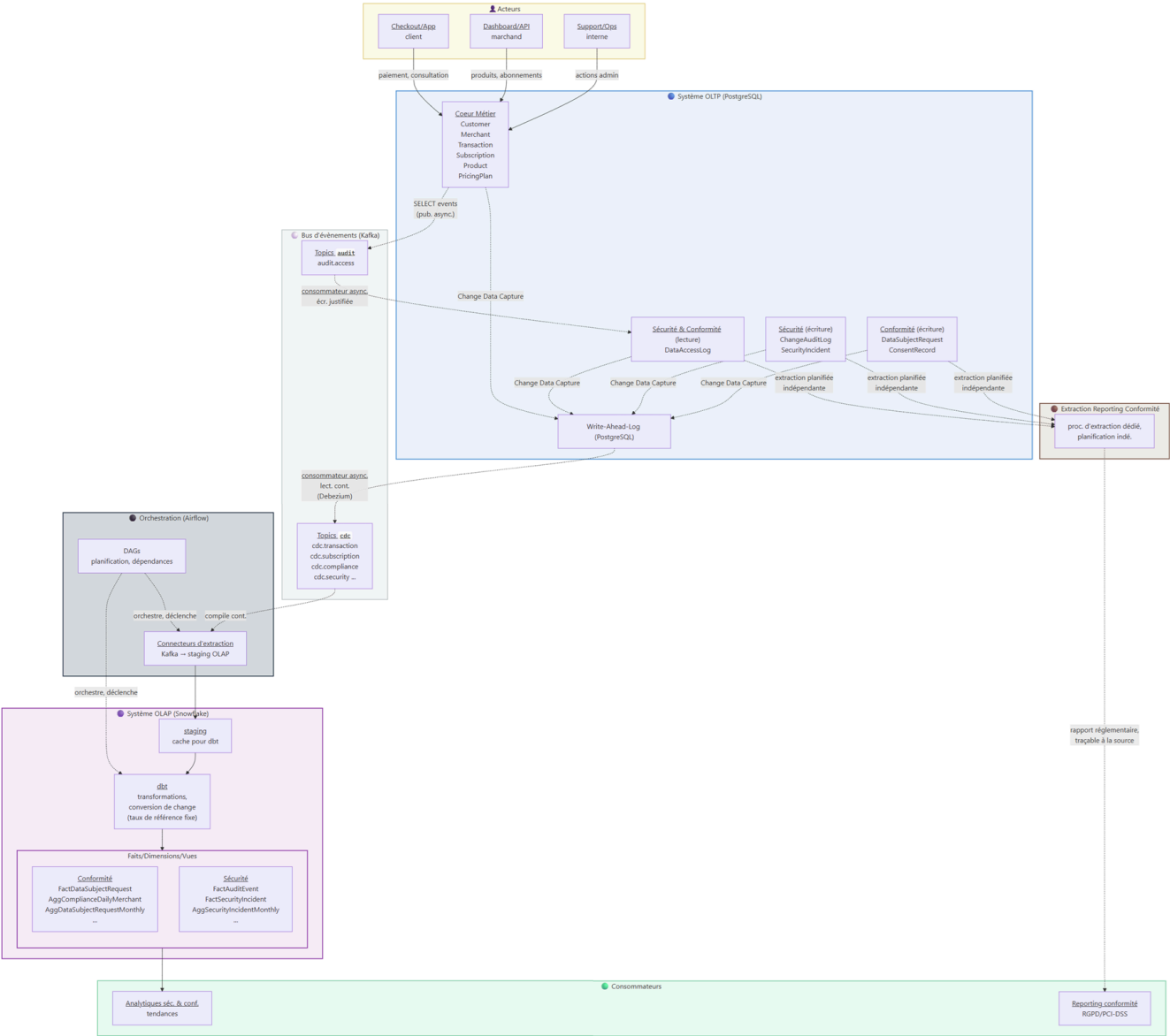
D1.2.2. Vue du coeur métier



D1.2.3. Vue du scoring de fraude



D1.2.4. Vue du reporting de sécurité et de conformité



D1.3. Lecture des flux de données

Flux	Mécanisme	Justification
client/marchand → OLTP	Écriture sync. directe	Chemin transactionnel, ACID requis (BR1)
OLTP Coeur → Topic cdc	WAL + Debezium	Même principe que <code>ChangeAuditLog</code> [D2-C] : jamais de trigger synchrone sur <code>Transaction</code>
OLTP Coeur → Topic app	Publication async. d'événement	<code>SELECT</code> sur OLTP Coeur ne laisse aucune trace dans WAL - rien à capter en CDC pour créer la ligne [D2-C]
Topic app → OLTP <code>DataAccessLog</code>	Consommateur async.	Le consommateur porte le contexte métier (justification) que seule l'application connaît au moment de l'accès
OLTP Sec & Conf → Topic cdc	WAL + Debezium	Une fois la ligne insérée dans OLTP (par le consommateur pour <code>DataAccessLog</code>), CDC la réplique comme n'importe quelle autre table
Topic app → NoSQL	Ingestion continue	Absorbe logs, clickstream, interactions - schéma flexible, volumétrie non bornée
Topic cdc → NoSQL merchants	Consommateur async.	Cache dénormalisé actualisé à chaque changement de OLTP Coeur (§D4.8 [D4]); évite d'interroger OLTP à chaque événement app
Topic cdc → NoSQL features	Déclenchement async.	Le recalcul des features suit le même flux CDC que "NoSQL merchants" ; pas de mécanisme distinct
NoSQL features → service scoring	Lecture temps réel	Alimente le modèle de fraude en continu
NoSQL merchants → service scoring	Lecture sync.	Seuils de risque par marchand, consultés en même temps que les features
Service de scoring → OLTP (décision)	Réponse temps réel, hors Kafka	Autorise/bloque la transaction ; latence bout en bout < 100 ms ([D7] §D7.2) ; la persistance du score est un flux séparé
Service de scoring → Topic fraud	Publication async.	Seul flux qui revient vers OLTP par un mécanisme asynchrone ([D5] §D5.5.4), distinct de la décision de fraude elle-même
Topic fraud → OLTP FraudScore et OLAP	Consommateur async. + dbt (orch. / Airflow)	Enregistre le score dans OLTP et alimente <code>FactFraudEvent</code> via dbt
Topic cdc + NoSQL → dbt (orch. / Airflow) → OLAP	Transform. planifiée	Conversion de change avec taux de référence fixe OLAP ; Airflow déclenche, dbt transforme et charge directement dans l'entrepôt OLAP ([D5] §D5.11)
OLAP → analytiques	Requêtes analytiques	Découplé du chemin transactionnel, pas d'impact sur latence de paiement, quatre consommateurs analytiques distincts, tous alimentés par OLAP ([D1] §D1.2) : 1 externe: marchand, 3 internes: revenu & produit, risque & fraude, sécurité & conformité
OLTP → reporting conformité	Extraction planifiée, non-Airflow	Alimente les rapports RGPD/PCI-DSS sans exposer l'OLTP directement aux outils de reporting ; job d'extraction dédié non intégré au pipeline partagé (pour supporter les pannes de Kafka/Airflow/dbt)

Remarques

`event_logs` **vs.** `audit.access`

`event_logs` est un journal d'erreur technique ou opérationnelle : une panne de service, un timeout, une exception applicative.
`audit.access` trace les accès en lecture des données OLTP.

	<code>event_logs</code>	<code>audit.access</code>
Question	"Qu'est-ce qui a mal fonctionné techniquement?"	"Qui a consulté quelle donnée?"
Déclencheur	Erreur/exception d'un service	Lecture réussie ou refusée
Exemple	"Card authorization timeout" sur <code>payment-gateway</code>	Un agent support consulte le profil d'un client
Alimente	Rien % sécurité/conformité	<code>DataAccessLog</code> (OLTP), alerting <i>Flink</i> (D5 §D5.8.5)

Reporting Sécurité & Conformité OLTP vs. OLAP

On a deux types de reporting lié à la *sécurité et à la conformité* :

	Reporting réglementaire	Reporting analytique
Consommateur	Délégué à la protection des données (DPO), responsable conformité	Analyste risque, équipe GRC (<i>Governance, Risk, Compliance</i>)
Produit	Dépôts légaux, notification de violation, réponse à une demande RGPD	Tableaux de bord de tendance, corrélations, revues périodiques
Contrainte	Délai légal opposable (72h, 30/45j), valeur probante	Aucune échéance légale, exploration libre
Fraîcheur	Nécessaire à l'instant du dépôt	Suffisante avec un décalage batch
Valeur	Traçabilité directe à la source, preuve d'audit	Corrélation avec d'autres dimensions (marchand, segment, temps)
Dépendance	Traçable à la source, isolé du pipeline partagé	Dépend du pipeline OLAP (Kafka/Airflow/dbt)
Source	OLTP	OLAP

Le chemin OLTP est volontairement minimal, aussi proche de la source que possible. D'un autre côté, l'outil de reporting RGPD/PCIDSS ne peut pas avoir de connexion directe à la base transactionnelle (cf. §D1.3 "alimente les rapports [...] sans exposer l'OLTP directement aux outils de reporting") . D'où la nécessité d'un job d'extraction dédié.

D1.4. Sujets non couverts

La sélection des technologies et des outils managés sous-jacents (*MongoDB Atlas*, self-hosted, ou service compatible de type *DocumentDB*) est une décision d'infrastructure et d'hébergement hors du périmètre de l'architecture de données.

La sélection du bus d'évènements *Kafka* est une exigence technique dans le *business case* [R0], bien qu'elle soit normalement exclue du périmètre de l'architecture de données proprement dite.

La sélection de l'entrepôt analytique *Snowflake* est une décision prise dans [D5] et justifiée dans [D5-A], mais représentée ici pour permettre au diagramme d'architecture de montrer les choix techniques les plus importants sans avoir à dépendre du document technique [D5].

La stratégie de réplication/failover propre à *OLTP* est détaillée séparément (cf. [D2-C]).

Références

Ressources

- [R0] [Stripe Business Case](#)

Livrables

- [D2] [OLTP Entity-Relation Diagrams](#)

- [D3] [OLAP System Schema Design](#)
- [D4] [NoSQL Data Model](#)
- [D5] [Data Pipeline Architecture](#)
- [D7] [Machine Learning Integration Strategy](#)

Annexes

- [D2-C] [OLTP Supporting Notes](#)
 - [D5-A] [Data Pipeline Supporting Notes](#)
-

Annexe : Diagramme d'architecture globale complet

